

END-TO-END DEVOPS & DEPLOYMENT GUIDE

Deploying CEO Portfolio Command Center from GitLab

Comprehensive step-by-step instructions for repository initialization, secrets management, CI/CD pipeline automation, Docker containerization, and production cloud hosting.

APPLICATION
FastAPI + Python 3.10

INTEGRATIONS
Google Sheets (gsread)

STORAGE
JSON & Automatic Backups

CI/CD ENGINE
GitLab CI/CD & Docker

ROADMAP

Deployment Execution Workflow

Phase	Action Item	Primary Outcome
Phase 1	GitLab Repository Setup & Initial Push	Push source code to GitLab without leaking private keys/secrets.
Phase 2	CI/CD Secrets & Variables Configuration	Store <code>credentials.json</code> , sheet IDs, and SSH keys in GitLab Vault/Variables.
Phase 3	Automated Pipeline (<code>.gitlab-ci.yml</code>)	Automated testing, container building, and deployment triggering.
Phase 4	Production Server / VM Deployment	Run the app with Docker, Persistent Volumes, Nginx Reverse Proxy & SSL.
Phase 5	Google Sheets & Service Account Verification	Verify Sheet sharing permissions and real-time synchronization.
Phase 6	Monitoring, Health Checks & Maintenance	Verify <code>/api/health</code> , automated backup rotation, and container auto-restart.

PHASE 1

GitLab Repository Setup & Initial Push

CRITICAL SECURITY REQUIREMENT: DO NOT COMMIT SENSITIVE FILES

Never commit `credentials.json` or your production `.env` with private tokens to git history. Verify that your `.gitignore` contains them before pushing.

1 Verify .gitignore Protection

Ensure your `.gitignore` file contains the following sensitive and runtime entries:

```
# Credentials and local environment
credentials.json
.env
*.key
*.pem

# Python artifacts
__pycache__/
*.pyc
.pytest_cache/

# Local database backups (persistent data)
data/backups/
```

2 Create a New Project on GitLab

1. Log in to your **GitLab** account (gitlab.com or self-hosted instance).
2. Click **New project** > **Create blank project**.
3. Name the project: `ceo-portfolio-dashboard` (or your preferred name).
4. Set Visibility Level to **Private** (recommended for enterprise portfolio data).
5. **Uncheck** "Initialize repository with a README" (since you already have a local codebase).
6. Click **Create project** and copy the repository URL (e.g. `https://gitlab.com/your-username/ceo-portfolio-dashboard.git` or `git@gitlab.com:your-username/ceo-portfolio-dashboard.git`).

3 Initialize Local Git & Push to GitLab

Open PowerShell or Terminal inside your project directory and execute:

```
# 1. Initialize local git repository (if not already done)
git init

# 2. Stage all project files
git add .

# 3. Create initial commit
git commit -m "Initial commit: CEO Portfolio Command Center application"

# 4. Rename default branch to main
git branch -M main

# 5. Add GitLab remote URL
git remote add origin https://gitlab.com/your-username/ceo-portfolio-dashboard.git

# 6. Push code to GitLab
git push -u origin main
```

PHASE 2 Configuring GitLab CI/CD Variables & Secrets

To keep the application secure and automated, sensitive configuration values are injected at runtime via GitLab CI/CD Variables.

1 Navigate to GitLab CI/CD Settings

In your GitLab repository, go to: **Settings** > **CI/CD** > **Variables** > **Expand** > **Add variable**.

2 Add Required CI/CD Variables

Variable Key	Type	Masked/Protected	Description / Value
<code>GOOGLE_SHEET_ID</code>	Variable	Protected	Your Google Spreadsheet ID (from the Google Sheets URL)
<code>GOOGLE_CREDENTIALS_JSON</code>	File	Protected	Paste the entire contents of your <code>credentials.json</code> service account file.
<code>SERVER_HOST</code>	Variable	Protected	Public IP or domain of your deployment server (e.g. <code>203.0.113.10</code>).
<code>SERVER_USER</code>	Variable	Protected	SSH username on production server (e.g. <code>deployer</code> or <code>ubuntu</code>).
<code>SSH_PRIVATE_KEY</code>	Variable	Masked + Protected	The private SSH key (OpenSSH format) used to authenticate to the production server.

PHASE 3 Creating the Automated CI/CD Pipeline (`.gitlab-ci.yml`)

Create a file named `.gitlab-ci.yml` in the root of your repository with the following complete automated pipeline configuration:

```

# =====
# GitLab CI/CD Pipeline: CEO Portfolio Command Center
# =====

stages:
  - test
  - build
  - deploy

# =====
# Stage 1: Automated Unit & Backend Tests
# =====

test:backend:
  stage: test
  image: python:3.10-slim
  before_script:
    - pip install --no-cache-dir -r requirements.txt
    - pip install pytest httpx
  script:
    - python -m pytest tests/ -v
  only:
    - main
    - merge_requests

# =====
# Stage 2: Build & Push Docker Container to GitLab Registry
# =====

build:docker:
  stage: build
  image: docker:24.0.5
  services:
    - docker:24.0.5-dind
  variables:
    DOCKER_IMAGE_TAG: $CI_REGISTRY_IMAGE:$CI_COMMIT_SHORT_SHA
    DOCKER_IMAGE_LATEST: $CI_REGISTRY_IMAGE:latest
  before_script:
    - docker login -u $CI_REGISTRY_USER -p $CI_REGISTRY_PASSWORD $CI_REGISTRY
  script:
    - docker build -t $DOCKER_IMAGE_TAG -t $DOCKER_IMAGE_LATEST .
    - docker push $DOCKER_IMAGE_TAG
    - docker push $DOCKER_IMAGE_LATEST
  only:
    - main

# =====
# Stage 3: Automated Continuous Deployment to Production Server via SSH
# =====

deploy:production:
  stage: deploy
  image: alpine:latest
  before_script:
    - apk update && apk add --no-cache openssh-client bash curl
    - eval $(ssh-agent -s)
    - echo "$SSH_PRIVATE_KEY" | tr -d '\n'
  ' | ssh-add -
    - mkdir -p ~/.ssh
    - chmod 700 ~/.ssh
    - ssh-keyscan -H $SERVER_HOST >> ~/.ssh/known_hosts
  script:
    - |
      ssh $SERVER_USER@$SERVER_HOST << 'EOF'

```

```
set -e
echo "→ Logging into GitLab Container Registry ..."
echo "$CI_REGISTRY_PASSWORD" | docker login -u "$CI_REGISTRY_USER" --password-stdin "$CI_REGISTRY"

echo "→ Pulling latest container image ..."
docker pull $CI_REGISTRY_IMAGE:latest

echo "→ Creating host storage directories ..."
mkdir -p /opt/ceo-dashboard/data /opt/ceo-dashboard/config

echo "→ Updating credentials and environment ..."
echo "$GOOGLE_CREDENTIALS_CONTENT" > /opt/ceo-dashboard/credentials.json
echo "GOOGLE_SHEET_ID=$GOOGLE_SHEET_ID" > /opt/ceo-dashboard/.env
echo "GOOGLE_CREDENTIALS_FILE=/app/credentials.json" >> /opt/ceo-dashboard/.env

echo "→ Restarting application container ..."
docker stop ceo-dashboard || true
docker rm ceo-dashboard || true

docker run -d --name ceo-dashboard --restart always -p 5000:5000

echo "→ Deployment finished successfully!"
EOF
environment:
  name: production
  url: http://$SERVER_HOST:5000
only:
  - main
```

PHASE 4

Production Server Provisioning & Hosting Options

Option A: Linux VPS / Cloud Server (AWS EC2 / DigitalOcean / Linode / GCP / Hetzner)

This is the standard, robust method for enterprise production hosting.

1 Server Setup & Docker Installation (Ubuntu 22.04 / 24.04 LTS)

SSH into your server and run:

```
# 1. Update system packages
sudo apt update && sudo apt upgrade -y

# 2. Install Docker & Docker Compose Plugin
curl -fsSL https://get.docker.com -o get-docker.sh
sudo sh get-docker.sh

# 3. Add deploy user to docker group
sudo usermod -aG docker $USER
newgrp docker

# 4. Create persistent application folders
sudo mkdir -p /opt/ceo-dashboard/data
sudo chown -R $USER:$USER /opt/ceo-dashboard
```

2 Configure Nginx Reverse Proxy with Free SSL (HTTPS)

Install Nginx and Certbot for secure HTTPS traffic on your custom domain:

```
# Install Nginx and Certbot
sudo apt install -y nginx certbot python3-certbot-nginx

# Create Nginx configuration for your domain
sudo nano /etc/nginx/sites-available/ceo-dashboard
```

Paste the following Nginx virtual host block:

```
server {
    server_name dashboard.yourcompany.com;

    client_max_body_size 50M;

    location / {
        proxy_pass http://127.0.0.1:5000;
        proxy_http_version 1.1;
        proxy_set_header Upgrade $http_upgrade;
        proxy_set_header Connection "upgrade";
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Enable the site and obtain an SSL certificate:

```
sudo ln -s /etc/nginx/sites-available/ceo-dashboard /etc/nginx/sites-enabled/
sudo nginx -t
sudo systemctl reload nginx

# Obtain Let's Encrypt SSL Certificate
sudo certbot --nginx -d dashboard.yourcompany.com
```

Option B: Docker Compose Setup (Standalone / Alternative)

If you prefer deploying via `docker-compose.yml` directly on the server:


```
version: '3.8'

services:
  dashboard:
    image: registry.gitlab.com/your-username/ceo-portfolio-dashboard:latest
    container_name: ceo_portfolio_app
    restart: unless-stopped
    ports:
      - "127.0.0.1:5000:5000"
    env_file:
      - .env
    volumes:
      - ./data:/app/data
      - ./credentials.json:/app/credentials.json:ro
    healthcheck:
      test: ["CMD", "curl", "-f", "http://localhost:5000/api/health"]
      interval: 30s
      timeout: 10s
      retries: 3
```

PHASE 5 Google Cloud Service Account & Sheets Sync Setup

1 Google Cloud Console Service Account Key

1. Go to **Google Cloud Console** (console.cloud.google.com).
2. Enable **Google Sheets API** and **Google Drive API** in your project.
3. Navigate to **IAM & Admin > Service Accounts** > Click **Create Service Account**.
4. Generate and download the **JSON Key**. Keep this file safe (`credentials.json`).

2 Share Google Sheet with Service Account

1. Open your Google Sheet (e.g. `Portfolio Master Tracker`).
2. Click the top-right **Share** button.
3. Paste the `client_email` address found in your `credentials.json` (e.g. `service-account@project.iam.gserviceaccount.com`).
4. Assign permissions: **Editor** (or **Viewer** if read-only) and click **Share**.
5. Copy the Sheet ID from the URL: `https://docs.google.com/spreadsheets/d/<SHEET_ID>/edit` .

PHASE 6 Verification, Health Checks & Maintenance

1 Verify Backend Health Endpoint

Test that the application is running, connected to Google Sheets, and initialized:

```
curl -s http://localhost:5000/api/health | jq .
```

Expected Response:

```
{
  "status": "healthy",
  "timestamp": "2026-08-26T16:50:00.000Z",
  "hasCredentials": true,
  "credentialsFile": "credentials.json",
  "configuredSheetId": "1fpSVA3Agdu1iinkUEbwBmPNLHRV3KLRcQckeUUEXAsI",
  "totalActions": 24,
  "totalDecisions": 12,
  "totalPriorities": 8
}
```

2 Check Live Docker Logs

```
# Stream live container logs
docker logs -f --tail=100 ceo-dashboard
```

3 Automated Data & Backup Persistence

Because `/opt/ceo-dashboard/data` is mounted to `/app/data`, all your portfolio data (`dashboard_data.json`) and automated historical backups (`data/backups/`) survive server reboots, container updates, and CI/CD redeployments.

DEPLOYMENT COMPLETE!

Every time you push new code to the `main` branch on GitLab, the CI/CD pipeline will automatically run tests, build a fresh Docker container, and deploy the updated application to your production server with zero data loss.